AI 辅助的社团财务与库存管理系统

使用尽量简洁的数据模型，并接入 AI 进行日常事务的文书工作。

“转移”作为一等公民

我们将引入复式记账的思路。

文中“账户”指的是任何能够存放资金/货物的实体（实际的或者抽象的），而记录都聚焦于“钱/东西在账户之间的转移”。

例如，小明垫付 100 元订购 10 个立牌，记下到货、垫付和报销三笔：



这个模型足够简洁，而且也被很多现实世界的管理系统应用。

最重要的是，系统越简单，**人类和大模型**在操作时因理解不足产生的错误操作/幻觉就会越少，十万个按钮/工具和一堆层层叠叠的字段依赖会把所有没经过专业训练的人类逼疯。

AI 辅助日常操作和计算

我们支持网页内调用模型，或者让你自己的 coding agent 连接 MCP 服务。网站有引导 agent 接入网站 MCP 的文档，给 AI 域名之后他们就会自动发现并接入所有功能。接入 AI 助手后，以小明这笔垫付款为例，AI 这样工作：

成员提出需求 → **AI 提交待审记录** → **管理员核对审批**

1. **小明说明情况** “我今天垫了 100 元，向厂家买了 10 个立牌，货还没到。帮我记一下这笔垫付款。”
2. **AI 整理并提交** AI 查询已有账户，把这句话整理成“**小明 → 厂家，100 元**”，附上日期和采购说明，提交。
3. **管理员核对并批准** 核对付款人、收款人、金额和用途后批准，系统里小明的账户余额就减少 100 元；若原来互不相欠，现在就表示社团应报销他 100 元。

对于所有 Agent（和不受信任的用户），他们的操作都需要“人类（受信任的用户）审批”，防止手滑抽风。当然，即使有一笔错误的转移，我们目前完全可以通过“追加一个方向相反的转移”来抵消掉它。

随时核对钱和货

公账 社团账面还有多少钱。

个人 成员应交回多少，社团应偿还多少。

库存 货物都存在哪，各处还剩多少货物。

厂家 付给这个厂家的钱：余额 $+100$ 元，即已付 100 元。

展会游客 从游客收到的钱：余额 -120 元，即已收 120 元。

“厂家”和“展会游客”是虚拟账户，只记录我们与对方的往来。

仓库：https://github.com/Renko6626/FGAS

订一批立牌，拿去展会卖

小明先垫 100 元订了 10 个立牌，全部拿去展会，每个卖 20 元。卖掉 6 个后，钱先收进小红的收款码，事后再转进公账；收摊时发现只剩 3 个，核对后确认丢了 1 个。剩下的带回仓库，最后给小明报销。

货物的转移

1 从厂家订货

厂家 —10 个立牌→ 仓库

仓库 $+10$；厂家记 -10，表示这 10 个已经发给我们了。

2 带去展会卖 6 个

仓库 —10 个→ 展会摊位

展会摊位 —6 个→ 展会游客

现在账面上仓库有 0 个，展会摊位有 4 个，展会游客有 6 个。

3 收摊发现只剩 3 个

清点后确认是真丢了 1 个，就把它记到“虚空”：

展会摊位 —1 个→ 虚空

这样账面就和实际对上了。

4 剩下的带回

展会摊位 —3 个→ 仓库

最后：仓库 $+3$，摊位 0，游客 $+6$（卖出），虚空 $+1$（遗失）。

钱的转移

1 小明订货付钱

小明 —100 元→ 厂家

小明账户变成 -100，代表社团应该给他报销 100。**个人账户余额原则上应为 0：为 0 就是互不相欠。**

2 卖出 6 个，收到 120 元

假设收款码用的是小红的：

展会游客 —120 元→ 小红

此时小明账户 -100，小红账户 $+120$。

3 社团大报销

小红 —120 元→ 公账

公账 —100 元→ 小明

报销完成，两边的账都结清了。

最后：小明、小红账户 0（没人欠钱），公账 $+20$（结余），展会游客 -120（收入），厂家 $+100$（成本）。